Functions - inputs, results, and behavior — review

Exercise 0 — Return and use a value

Write `triple(number)`. Save its result for `4`, then print that result plus `2`. Explain which line returns and which displays.

```
def triple(number):
    # YOUR CODE HERE
    pass
```

Exercise 0 — Return and use a value — answer

```
1 def triple(number):
2     return number * 3
3 result = triple(4)
4 print(result + 2)
5 assert result == 12
```

EXERCISE

Exercise 1 — Rewrite a side effect

Rewrite `welcome` to return its message. Print from the caller. Explain why the original function is impure.

```
def welcome(name):
    print("Welcome " + name)
# YOUR CODE HERE
```

Exercise 1 — Rewrite a side effect — answer

```
1 def welcome(name):
2     return "Welcome " + name
3 print(welcome("Jenna"))
4 assert welcome("Jenna") == "Welcome Jenna"
```

Exercise 2 — Repeated input or repeated application

Does adding one have idempotence? Compare `add_one(3)` with `add_one(add_one(3))`. Explain why two calls with input 3 answer a different question.

```
1 def add_one(number):
2     return number + 1
3 # YOUR CODE HERE
```

Exercise 2 — Repeated input or repeated application — answer

```
1 def add_one(number):
2     return number + 1
3 print(add_one(3))
4 print(add_one(add_one(3)))
5 assert add_one(3) != add_one(add_one(3))
```

EXERCISE

Exercise 3 — Trace a returned function

Create adders for 2 and 10, then call both with 5. Explain what each remembers and why the factory is higher-order.

```
def make_adder(amount):
    def add(number):
        return number + amount
    return add
# YOUR CODE HERE
```

Exercise 3 — Trace a returned function — answer

```
def make_adder(amount):
    def add(number):
        return number + amount
    return add
add_two = make_adder(2)
add_ten = make_adder(10)
print(add_two(5))
print(add_ten(5))
assert add_two(5) == 7
assert add_ten(5) == 15
```